

Airline Data Query & Analysis

with Spark & HDFS

Matthieu Comparetto

University of Rome Tor Vergata
Sistemi e Architetture per Big Data

June 2026

Analyze 2.2 million flight records (Jan-Apr 2025) using a distributed approach.

- **Infrastructure:** Docker Compose simulating a multi-node cluster (HDFS & Spark).
- **Data Pipeline:** Typed Java manipulations using the Spark Dataset API
- **Machine Learning:** Unsupervised segmentation of carriers using MLlib (K-Means).

Query 1: Methodology & Implementation

Goal: Compare American Airlines (AA) and Delta (DL) performance.

```
// 1. Isolate and cache to avoid re-reading HDFS
Dataset<Row> filtered = flights.filter(...).cache();

// 2. Branch A: Average Delays (excluding cancellations)
Dataset<Row> delayStats = nonCancelled
    .groupBy("MONTH", "OP_UNIQUE_CARRIER")
    .agg(avg("DEP_DELAY").alias("dep_delay_mean"));

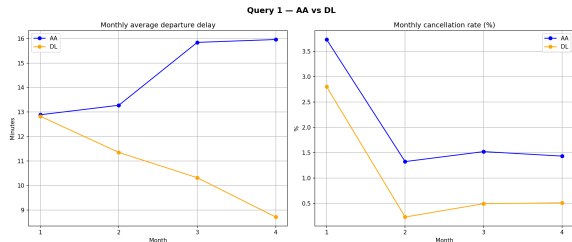
// 3. Branch B: Cancellation Rates
Dataset<Row> cancRate = filtered
    .groupBy("MONTH", "OP_UNIQUE_CARRIER")
    .agg(sum(col("CANCELLED")).divide(count("*")).multiply(100));

// 4. Merge results
Dataset<Row> result = delayStats.join(cancRate, ... "inner");
```

Query 1: Results & Trends

Operational Insights:

- **Diverging Trends:** Delta (DL) consistently reduced delays (13m $\rightarrow$ 9m), while American (AA) experienced increasing delays.
- **Weather Impact:** Severe cancellation peak in Month 1 (3.7% for AA).



Query 2: Methodology & Implementation (1)

Goal: Identify the Top 10 delayed airlines and their primary failure modes.

```
// 1. Filter out cancelled and diverted flights
Dataset<Row> validFlights = df.filter(
  col("CANCELLED").equalTo(0).and(col("DIVERTED").equalTo(0))
);

// 2. Stats aggregation
Dataset<Row> stats = validFlights
  .groupBy("OP_UNIQUE_CARRIER")
  .agg(
    count("*").alias("total_flights"),
    avg("ARR_DELAY").alias("avg_arr_delay"),
    avg("CARRIER_DELAY").alias("avg_carrier_delay"),
    avg("WEATHER_DELAY").alias("avg_weather_delay"),
    avg("LATE_AIRCRAFT_DELAY").alias("avg_late_aircraft_delay")
  );
```

Query 2: Methodology & Implementation (2)

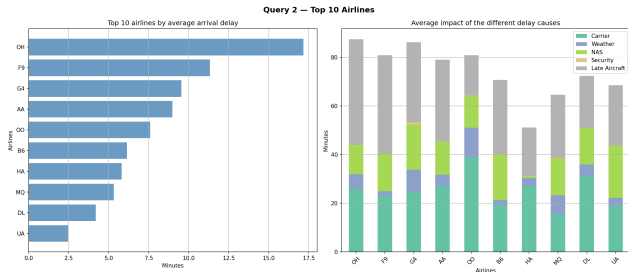
Goal: Identify the Top 10 delayed airlines and their primary failure modes.

```
// 3. Statistical relevance filter and Top 10 extraction  
Dataset<Row> result = stats  
    .filter(col("total_flights").geq(500))  
    .orderBy(col("avg_arr_delay").desc())  
    .limit(10);
```

Query 2: Results

Key Findings:

- **Top Delay:** PSA Airlines (OH) leads with over 17 minutes of average arrival delay.
- **Primary Causes:** Across all top 10 carriers, "Late Aircraft" and "Carrier" delays vastly outnumber weather-related issues.



Query 3 : Methodology & Implementation (1)

Goal: Map the hourly "Snowball Effect" of departure delays.

```
// 1. Hour extraction via integer
Dataset<Row> withHour = validFlights.withColumn("hour",
    col("CRS_DEP_TIME").divide(100).cast(DataTypes.IntegerType));

// 2. Big Data Approximation
Dataset<Row> percentiles = withHour.groupBy("OP_UNIQUE_CARRIER", "hour")
    .agg(
        percentile_approx(col("DEP_DELAY"), lit(0.50), lit(1000)).alias("p50"),
        percentile_approx(col("DEP_DELAY"), lit(0.90), lit(1000)).alias("p90"),
        // (P25 and P75 calculated similarly)
    );
```

Query 3 : Methodology & Implementation (2)

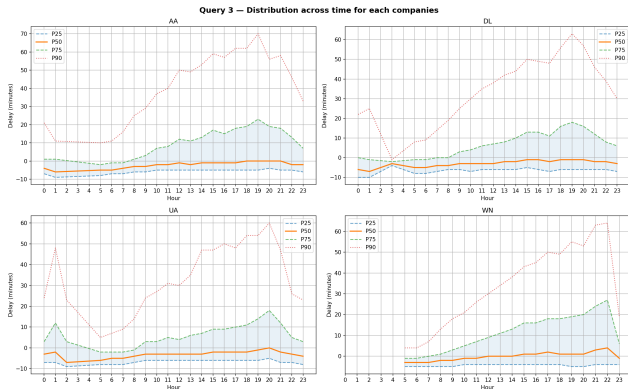
Goal: Map the hourly "Snowball Effect" of departure delays.

```
// 3. Separate aggregation for absolute delay bounds  
Dataset<Row> delay = validFlights.filter(col("CANCELLED").equalTo(0))  
    .groupBy("OP_UNIQUE_CARRIER")  
    .agg(min("DEP_DELAY"), max("DEP_DELAY"));
```

Query 3: Results

Hourly Distribution Insights:

- **Median Stability:** The P50 (median) remains low and relatively flat throughout the day, meaning most flights are on time.
- **Evening Congestion:** The 90th percentile (P90) demonstrates an exponential explosion starting around 18:00.
- **Data Pipeline Output:** The queries successfully fork the DAG to export both the hourly distributions and the absolute delay bounds into separate HDFS directories.



Query 4 : Methodology & Implementation (1)

Goal: Unsupervised carrier segmentation using Spark MLlib.

```
// 1. Grouping 8 operational metrics
VectorAssembler assembler = new VectorAssembler()
    .setInputCols(new String[]{
        "avg_arr_delay", "cancellation_rate", "avg_late_aircraft_delay", ...
    })
    .setOutputCol("features").setHandleInvalid("skip");
Dataset<Row> assembled = assembler.transform(stats);

// 2. Normalization Equalizing scale impact
StandardScaler scaler = new StandardScaler()
    .setInputCol("features").setOutputCol("scaled_features")
    .setWithMean(true).setWithStd(true);
Dataset<Row> scaledData = scaler.fit(assembled).transform(assembled);
```

Query 4 : Methodology & Implementation (2)

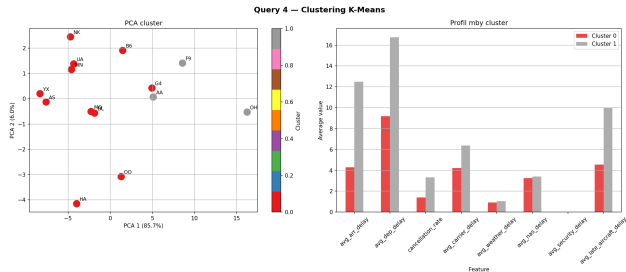
Goal: Unsupervised carrier segmentation using Spark MLlib.

```
// 3. Unsupervised Learning with K-Means Clustering  
KMeans kmeans = new KMeans()  
    .setK(2).setFeaturesCol("scaled_features").setSeed(42);  
Dataset<Row> predictions = kmeans.fit(scaledData).transform(scaledData);
```

Query 4: Carrier Clustering

PCA Projection & Profiles:

- **Dimensionality Reduction:** PCA was utilized in Python to project the 8 original ML features into a 2D space for visualization.
- **Cluster Validation:** The fixed random seed (`setSeed(42)`) ensures reproducibility of the algorithm across executions.
- **Cluster 0:** "Reliable Carriers" demonstrating high stability, low cancellation rates, and minimal delays.
- **Cluster 1:** "Underperforming Carriers" penalized by delays.



System Architecture & Data Flow

End-to-End Distributed Pipeline Workflow

Storage Layer (HDFS)

- Local CSV data ingested into the NameNode.
- Distributed and partitioned across the DataNode filesystem.

Analytical Pipeline

- Out-of-core operations leveraging the typed Dataset API.
- Results computed in parallel and exported directly back to HDFS.

Processing Layer (Engine)

- Java application compiled into a single **JAR** via Maven.
- Distributed execution managed by 1 Spark Master and 2 Workers.

Visualization Layer

- Downstream Python script pulls aggregated CSVs from the cluster.
- Pandas and Matplotlib utilized for reporting and PCA projection.

System Performance Benchmarks

Table: Average Execution Times per Query (in milliseconds)

Query	Iteration 1 (ms)	Iteration 2 (ms)	Iteration 3 (ms)
Query 1	9978	2325	2282
Query 2	7663	1896	1589
Query 3	15557	6275	6616
Query 4	17575	7762	6255
Average	12693	4565	4186

- **JVM Warm-up Overhead:** Initial iterations show higher latencies due to ClassLoading and Just-In-Time (JIT) compilation.
- **Optimal Steady-State:** Iteration 3 showcases the true throughput once Spark's execution graph (DAG) and memory caching are fully stabilized.

Conclusion & Future Work

Project Key Takeaways:

- Successfully simulated a multi-node production environment locally on a standalone development machine.
- Validated the operational efficiency and type-safety constraints of the Spark Dataset API over 2.2 million rows.

Future Architectural Evolutions:

- **Storage Layer Optimization:** Transition from raw flat CSVs to compressed columnar formats like **Apache Parquet** to minimize I/O bounds.
- **Physical Scalability:** Migrate the setup from Docker to a bare-metal physical multi-node cluster to evaluate real network constraints.
- **Advanced Orchestration:** Replace the static bash script with an enterprise-grade tool like **Apache Airflow** for robust ETL tracking.

Thank You

Any Questions?

Matthieu Comparetto
commatthieu@gmail.com

Sistemi e Architetture per Big Data — June 2026